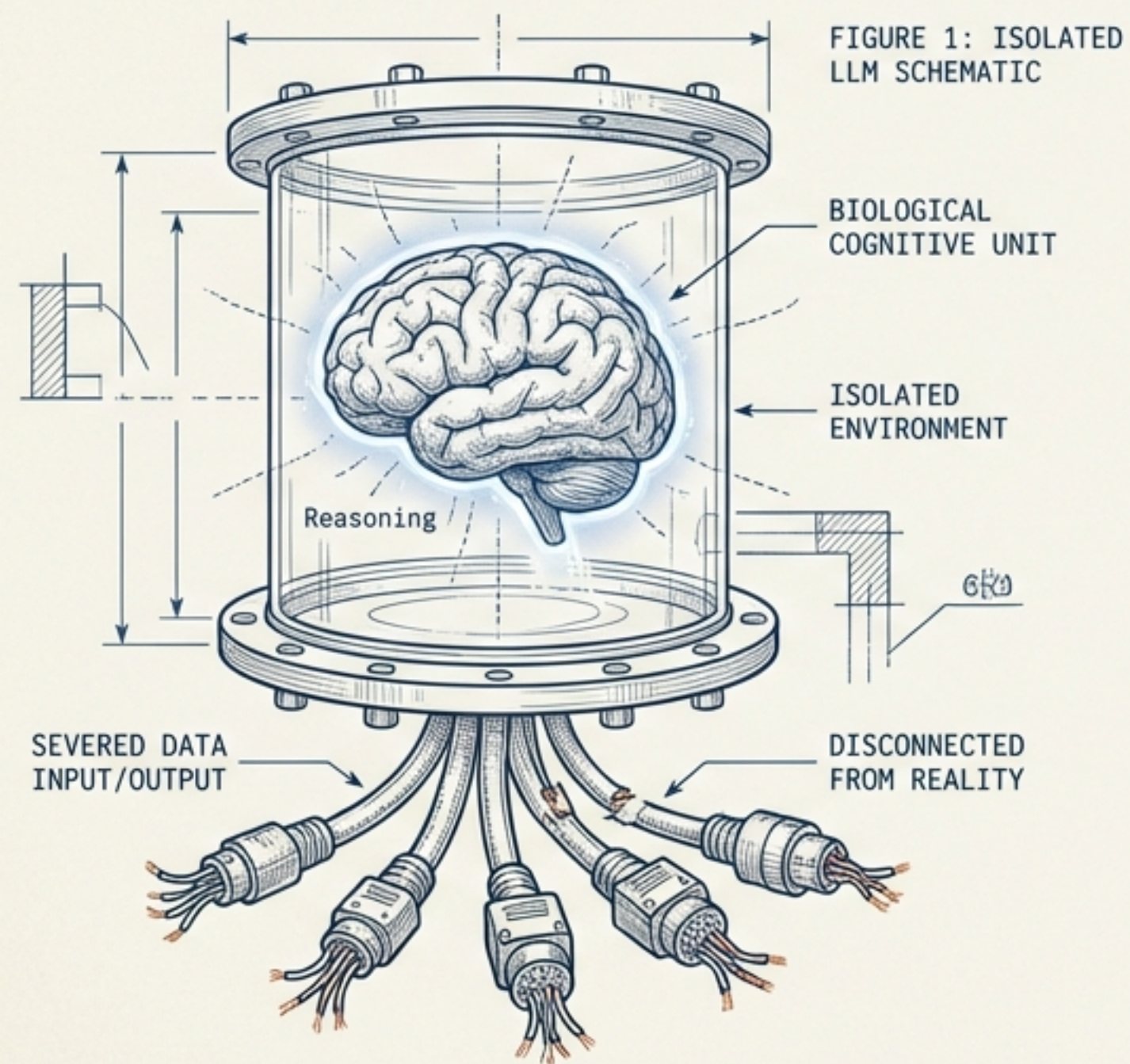


The Brain in a Vat: Understanding the Limits of Raw LLMs

A cognitive engine disconnected from the physical world.



TECHNICAL SPECIFICATIONS

Capabilities (The Brain)

💡	Reasoning: Deconstructing complex queries.
💬	Output Generation: Accessing parametric knowledge to speak.

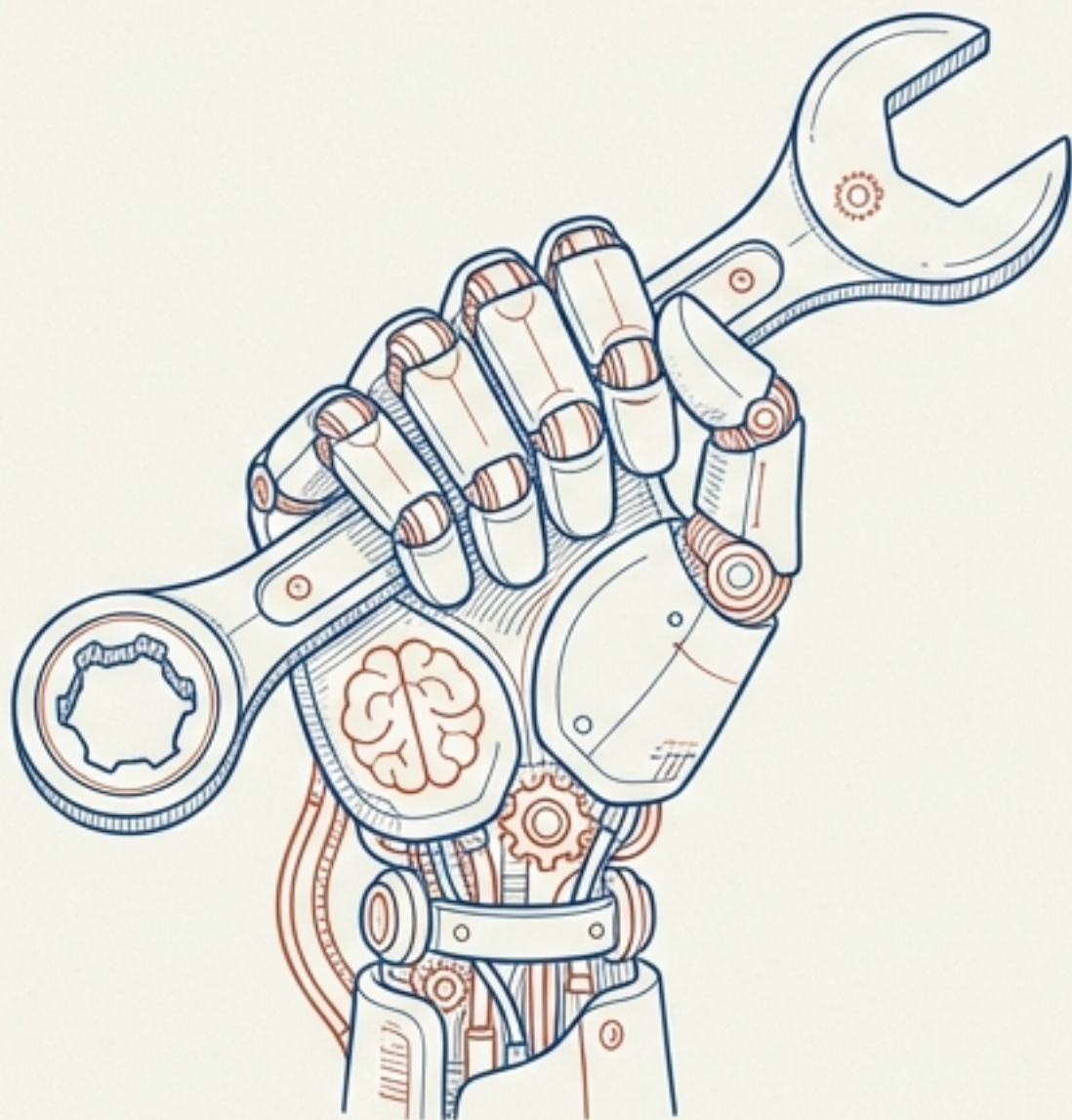
Limitations (The Disconnect)

🗄️❌	Cannot modify external databases.
🌐❌	Cannot post to social platforms.
☁️❌	Cannot query live APIs (e.g., Weather).

To transform an LLM from a passive thinker into an active participant, we must provide 'hands'. In LangChain, these hands are called **Tools**.

What is a Tool?

Bridging the gap between **natural language** and **executable code**.



The Mechanism

A function the Model knows how to discuss.

```
def multiply(a, b):  
    return a * b
```

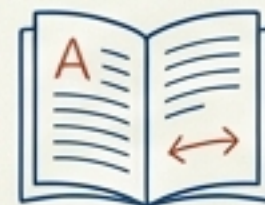
The Python Code

Name



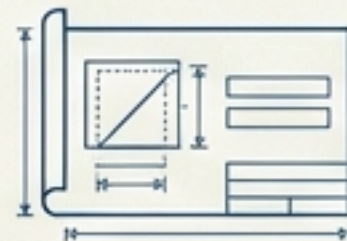
The Identifier (e.g., 'multiply')

Description



The User Manual. Tells the LLM *when* to use it.

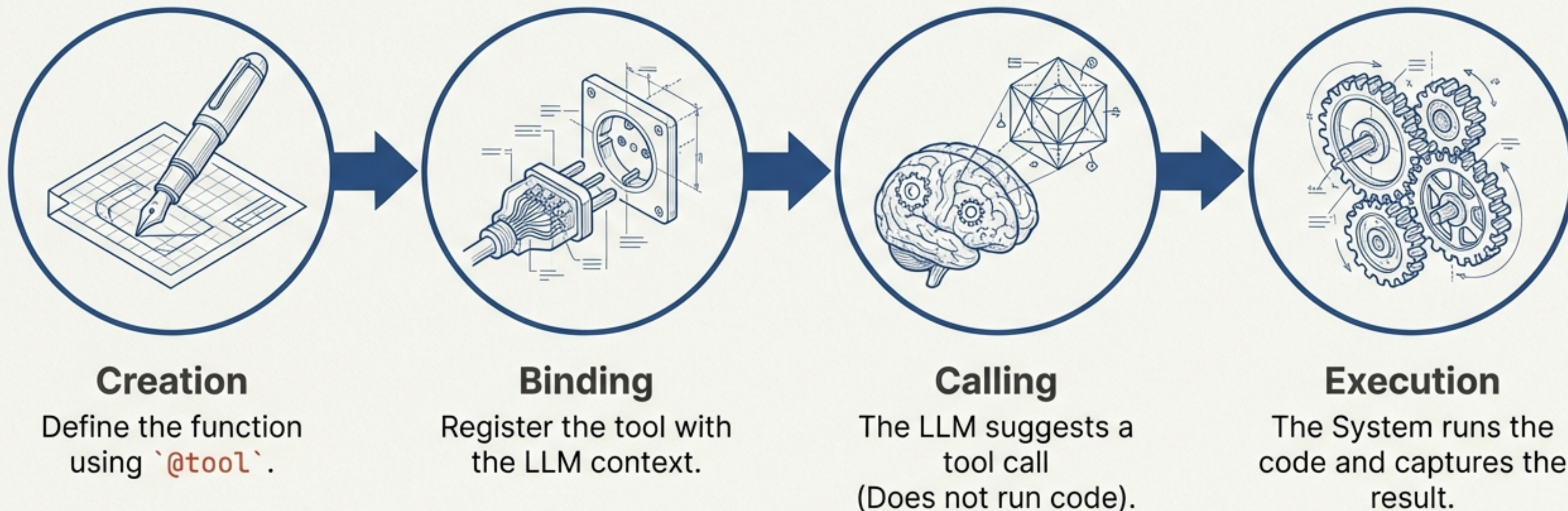
Schema



The Blueprint. Tells the LLM *how* to use it (Input Arguments).

The Tool Calling Lifecycle

From definition to execution: The four pillars of the process.



This cycle transforms a user query into an actionable outcome.

Step 1: Creation

Teaching the code to speak to the Model.

```
from langchain_core.tools import tool
```

```
@tool
```

```
def multiply(a: int, b: int) -> int:
```

```
    """Given two numbers a and b, this  
    tool returns their product.  
    """
```

```
    return a * b
```

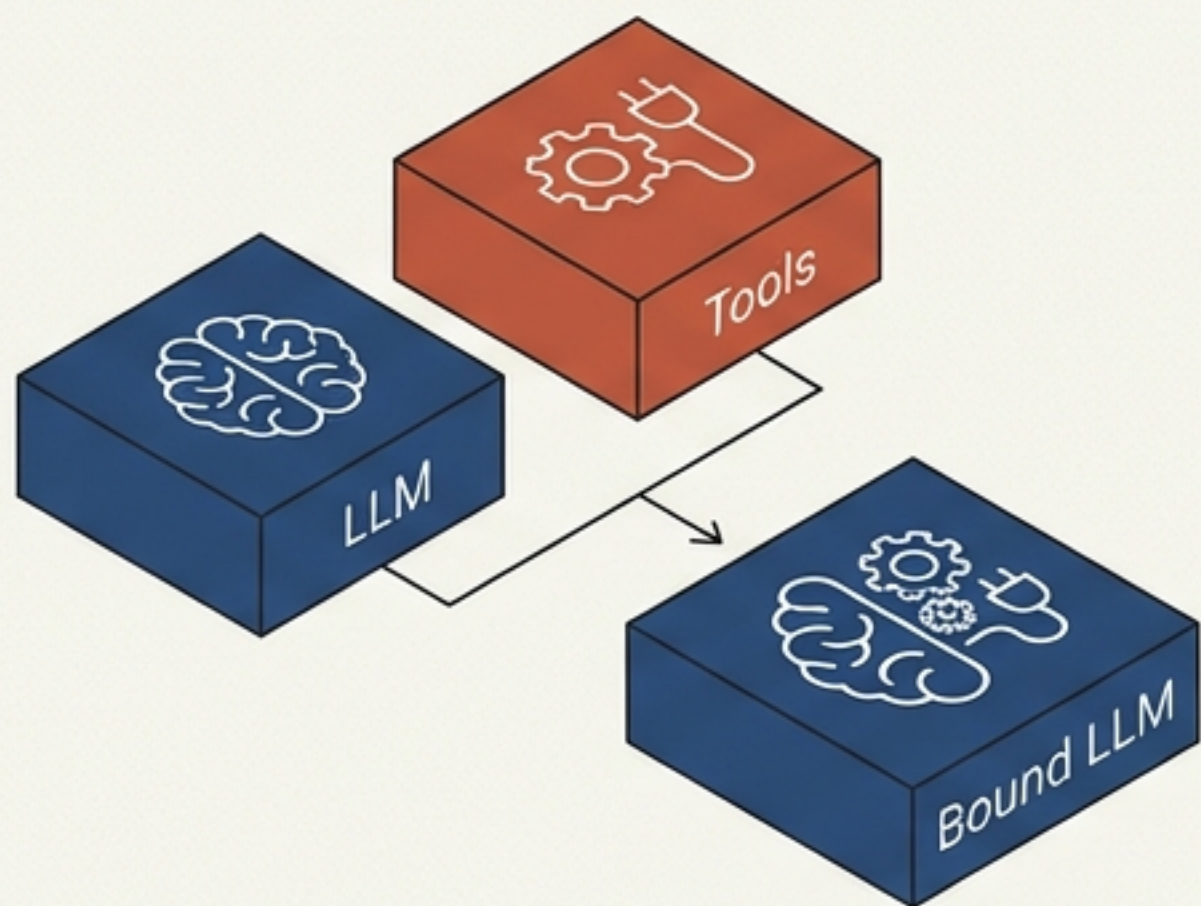
****The Decorator**:** Converts a standard function into a LangChain Runnable.

****The Prompt**:** This is crucial. It tells the LLM exactly *when* this tool is relevant.

****Type Hinting**:** Defines the JSON schema for the output generation.

Inspector View

```
inspect(multiply.args)  
-> {  
    'a': 'integer',  
    'b': 'integer'  
}
```



```
llm = ChatOpenAI(model="gpt-3.5-turbo")

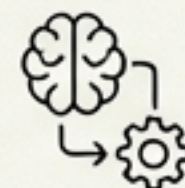
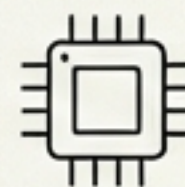
# The Registration Step
llm_with_tools = llm.bind_tools([multiply])
```

Step 2: Binding

Equipping the Brain with the Toolkit.

Key Information Points

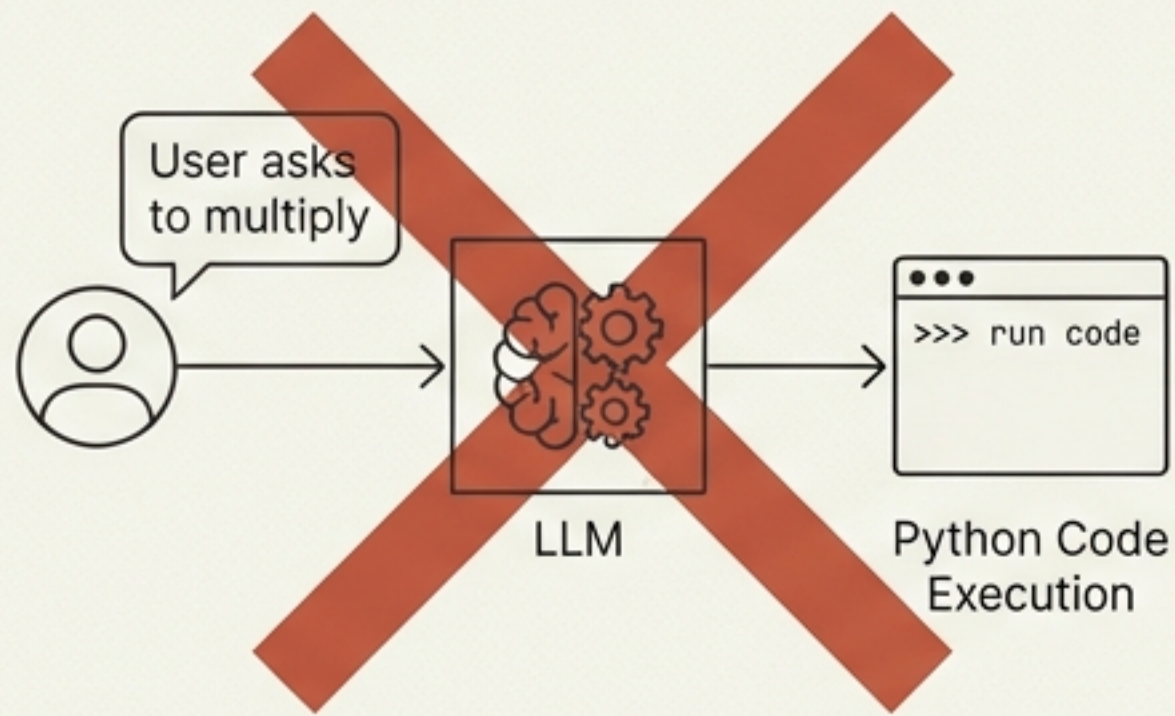
- **Compatibility:** Requires models fine-tuned for tool calling (e.g., GPT-3.5, GPT-4).
- **Mechanism:** `bind_tools` **does not execute code.** It injects the tool's signature (Name, Description, Schema) into the model's context.
- **Result:** The model is now "aware" of `multiply` and knows how to structure a request for it.



Step 3: Tool Calling

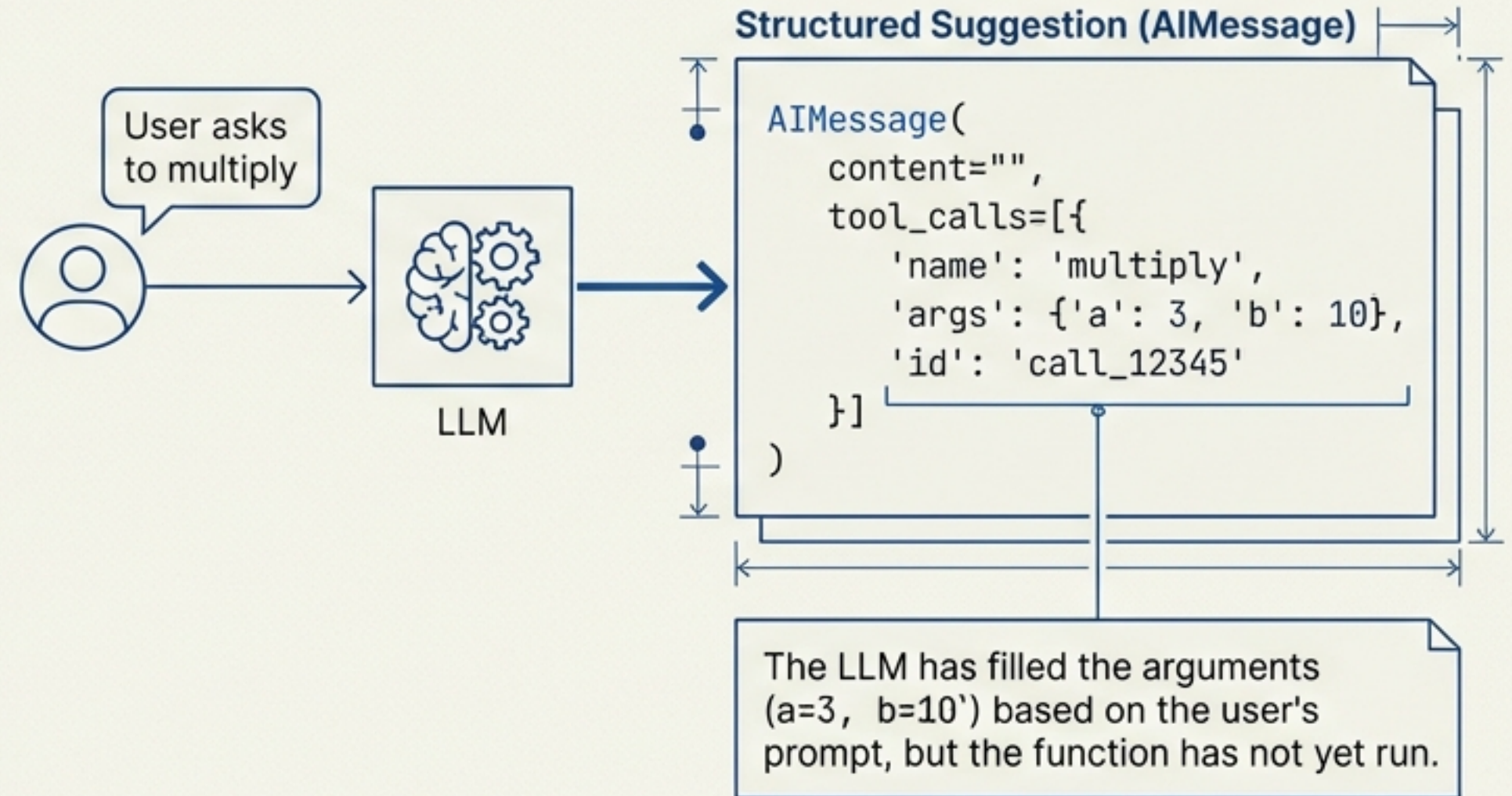
The Model acts as a consultant, not an executor.

The Myth



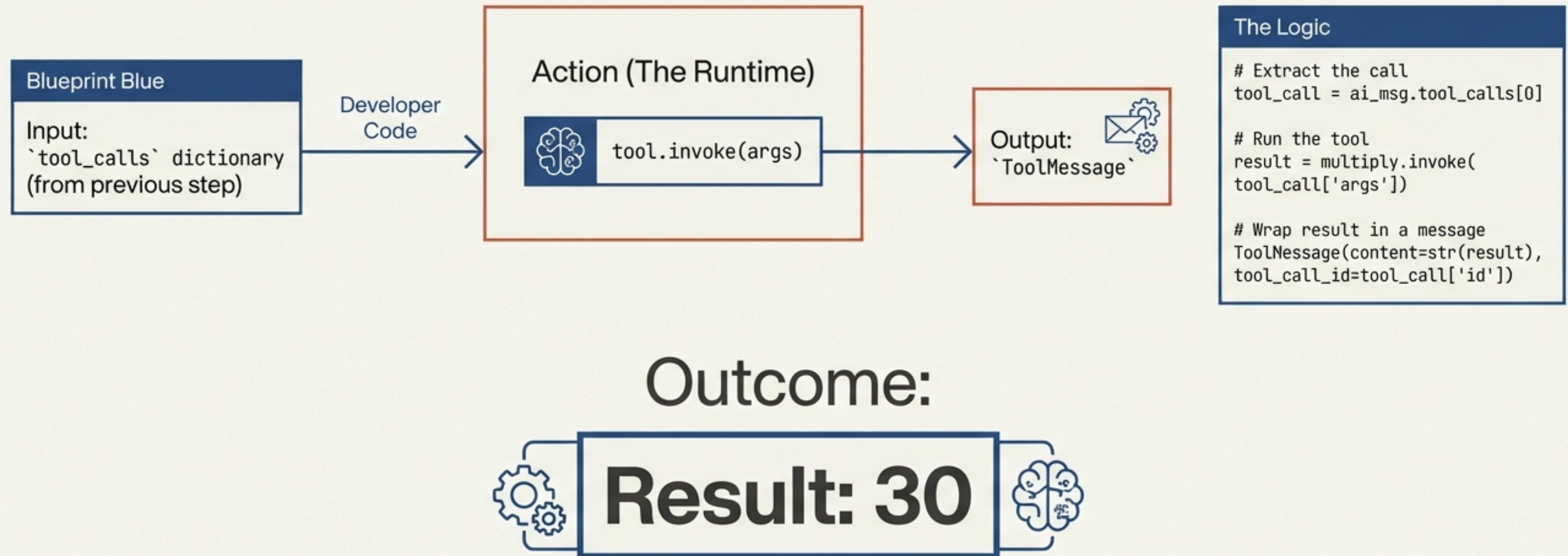
FALSE. The LLM cannot execute Python.

The Reality



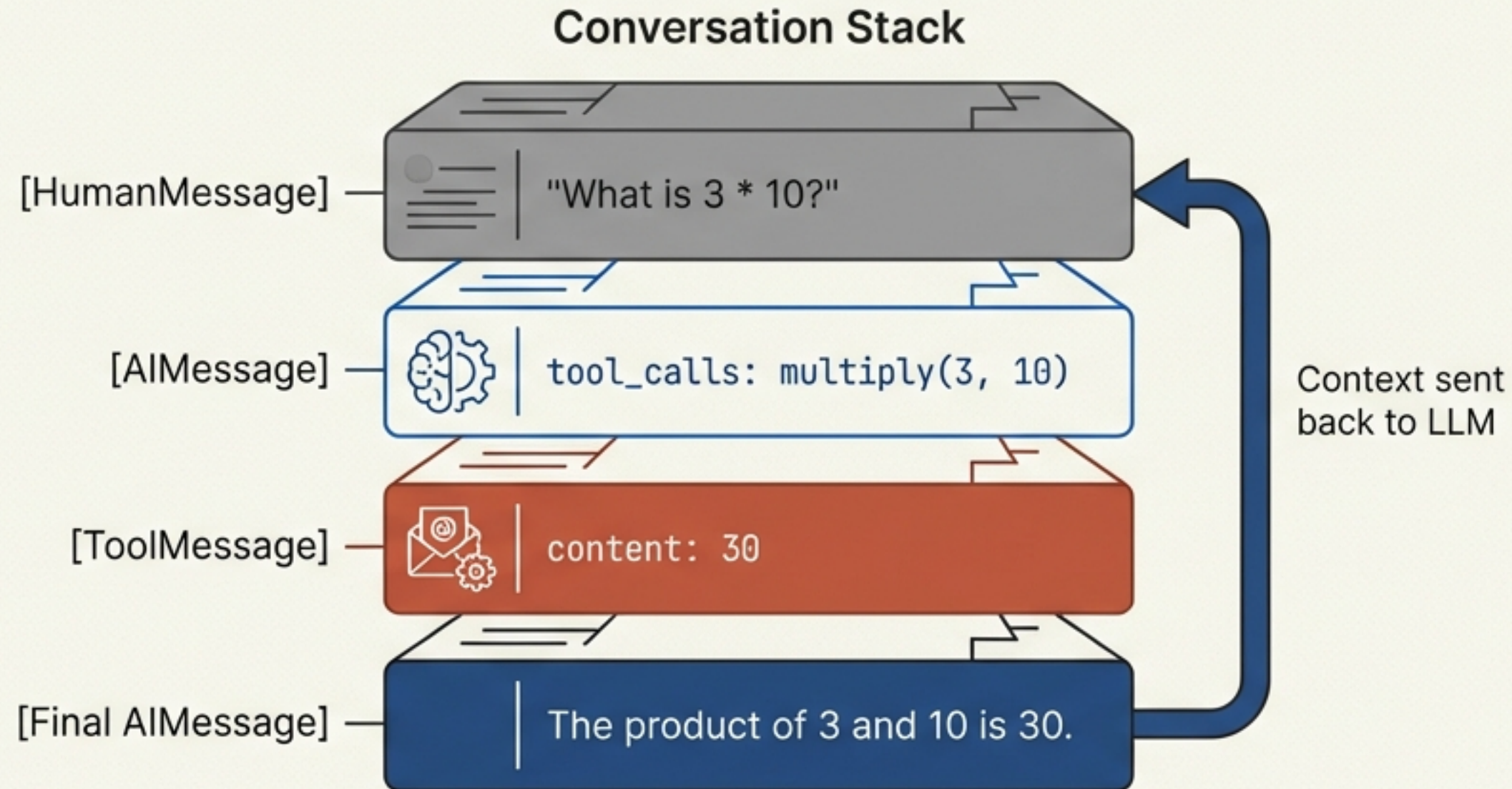
Step 4: Execution

The developer takes control to perform the action.

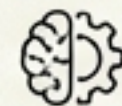


Closing the Loop

Synthesising data into a human answer.



Key Takeaway



The LLM receives the tool's raw output (30) and uses its "voice" to generate the final natural language response.

Application: Real-Time Currency Conversion

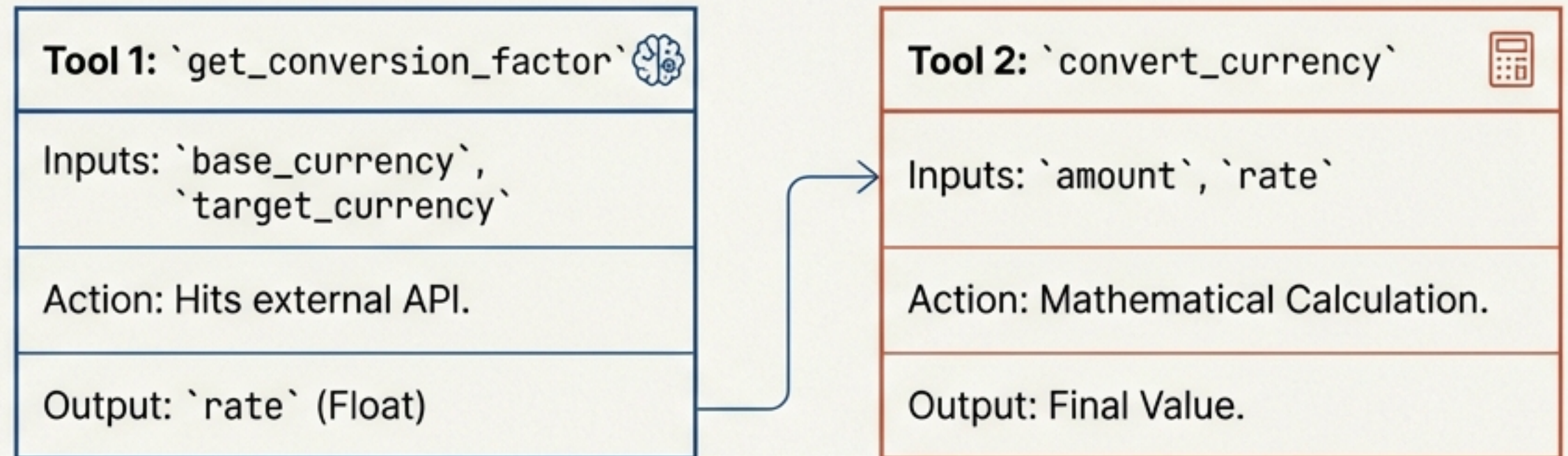
Chaining multiple tools to solve complex problems

The Scenario

User Query:
"Convert 80 USD to INR."

The Challenge:
LLMs are frozen in time.
They do not know today's
exchange rate.

The Architecture (Diagram)

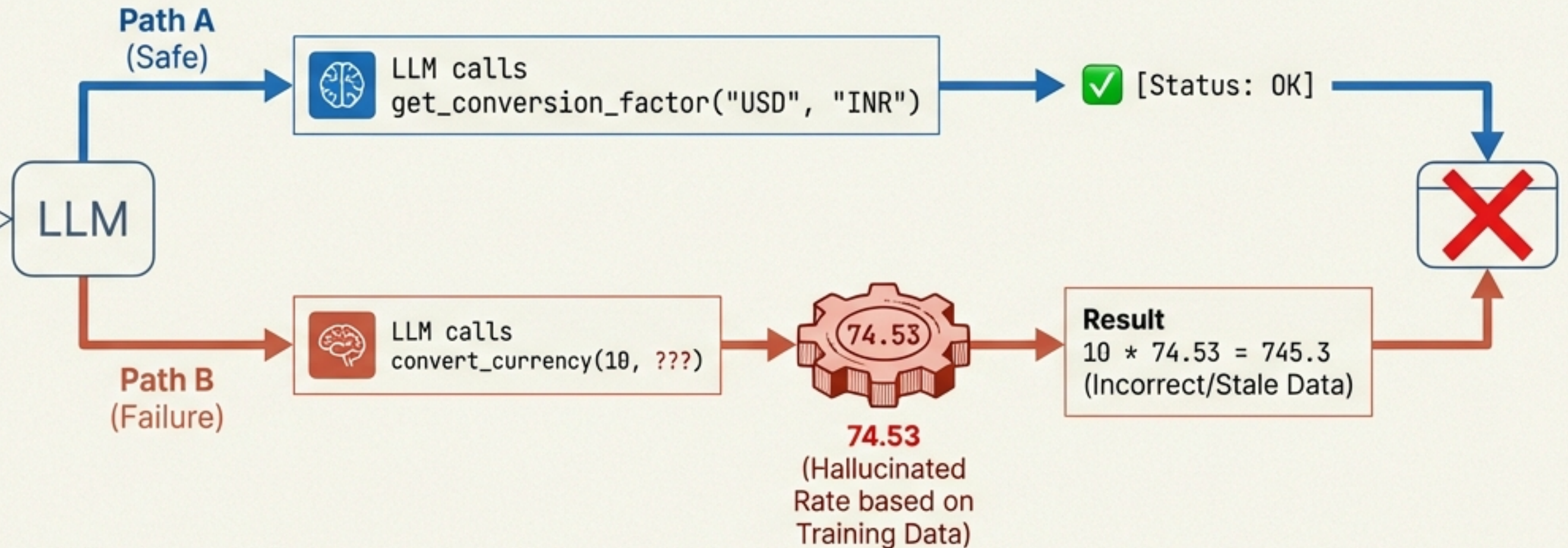


The Hallucination Trap

What happens when the Model guesses too early?

The Scenario

User asks: "What is the rate for USD to INR, and what is 10 USD in INR?"



⚠ Alert Box

The LLM attempts to run both tools in parallel. It hallucinates the missing 'rate' argument to satisfy the schema.

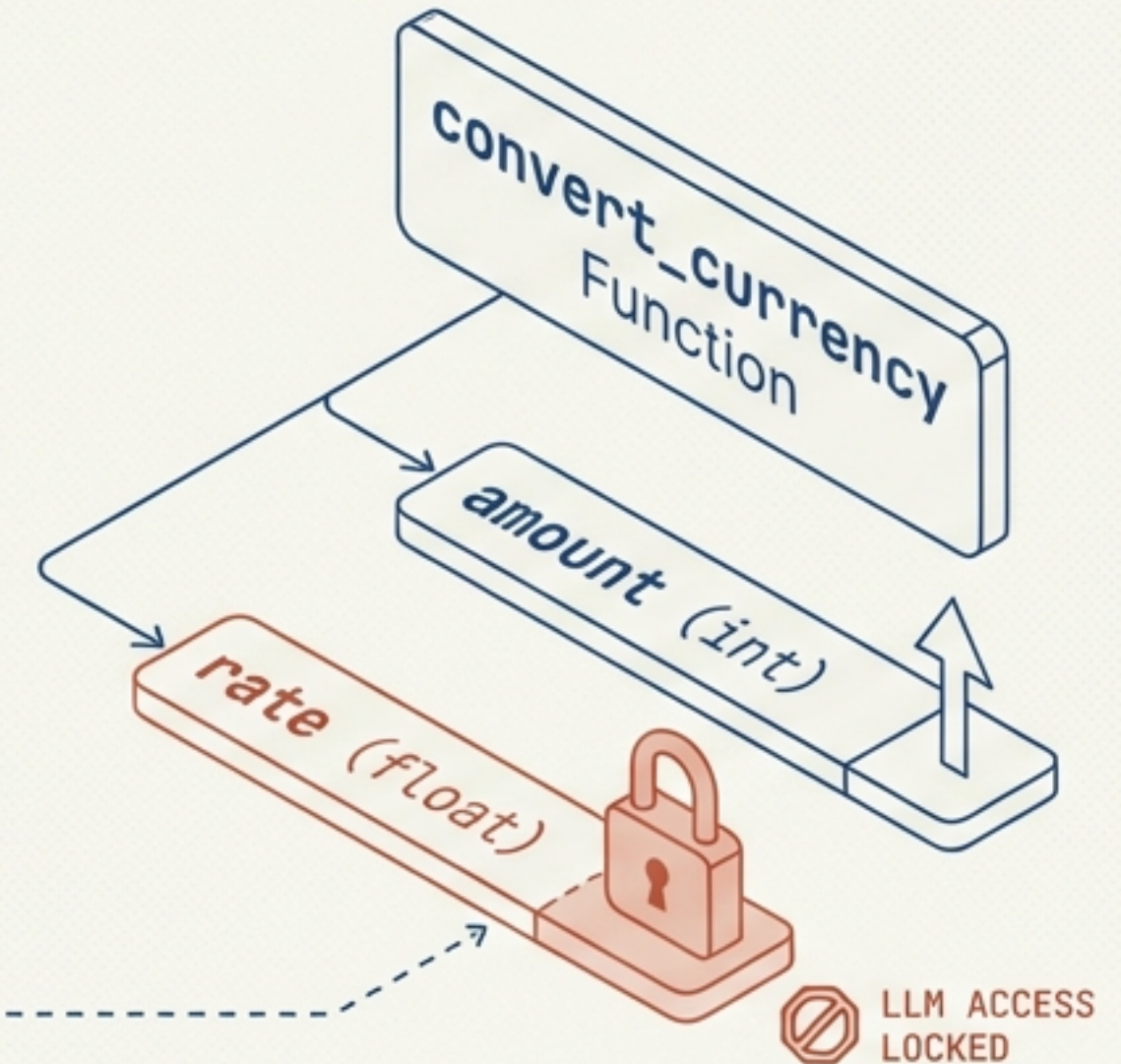
The Fix: InjectedToolArg

Forcing a sequence by restricting the LLM's access.

We explicitly forbid the LLM from generating the `rate` argument. We tell it: "Do not guess this. I will provide it later."

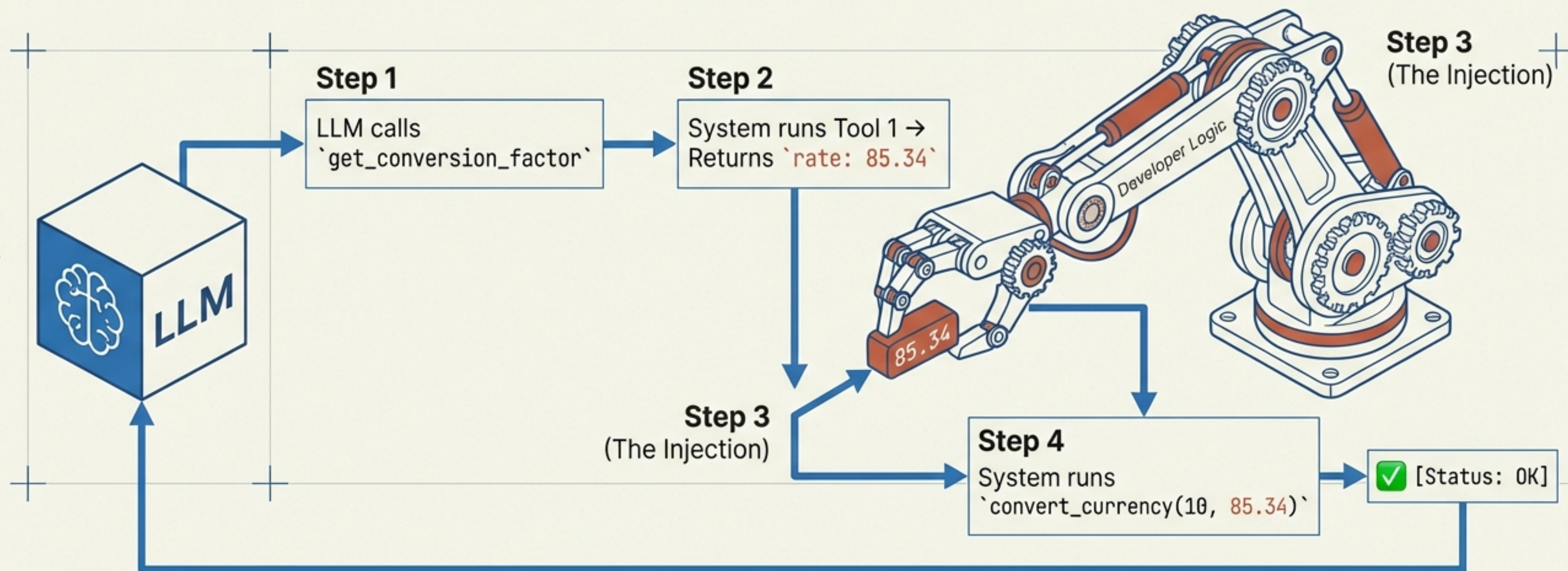
```
from langchain_core.tools import InjectedToolArg
from typing import Annotated

def convert_currency(
    amount: int,
    # The 'Stop Sign' for the LLM
    rate: Annotated[float, InjectedToolArg]
):
    ...
```



Orchestrating the Sequence

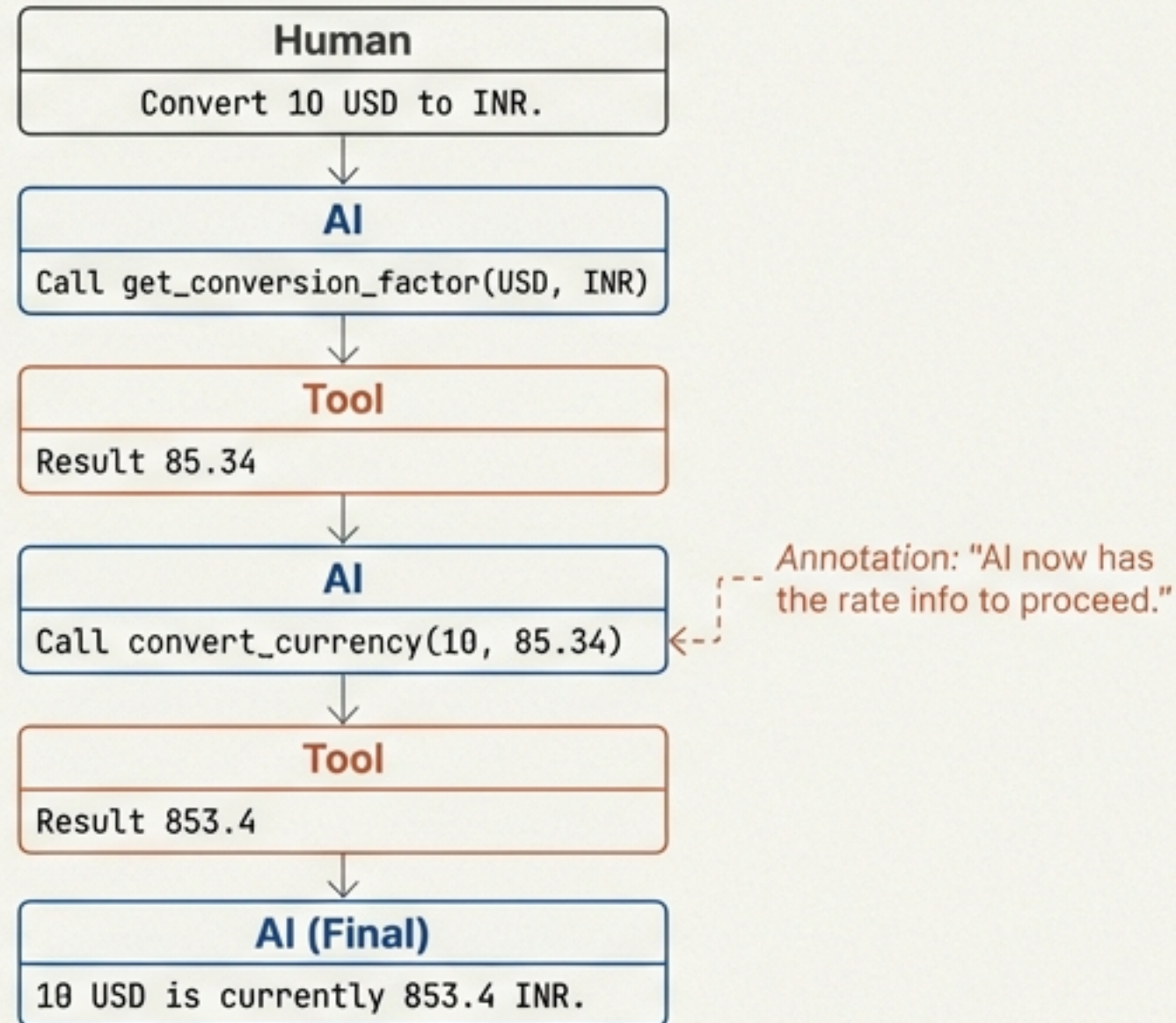
Manual control flow handles the dependency.



Note: The data flows from Tool 1 Output to Tool 2 Input, mediated by the Python script, not the LLM.

The Complete Context Stack

A transparent view of the final conversation history.



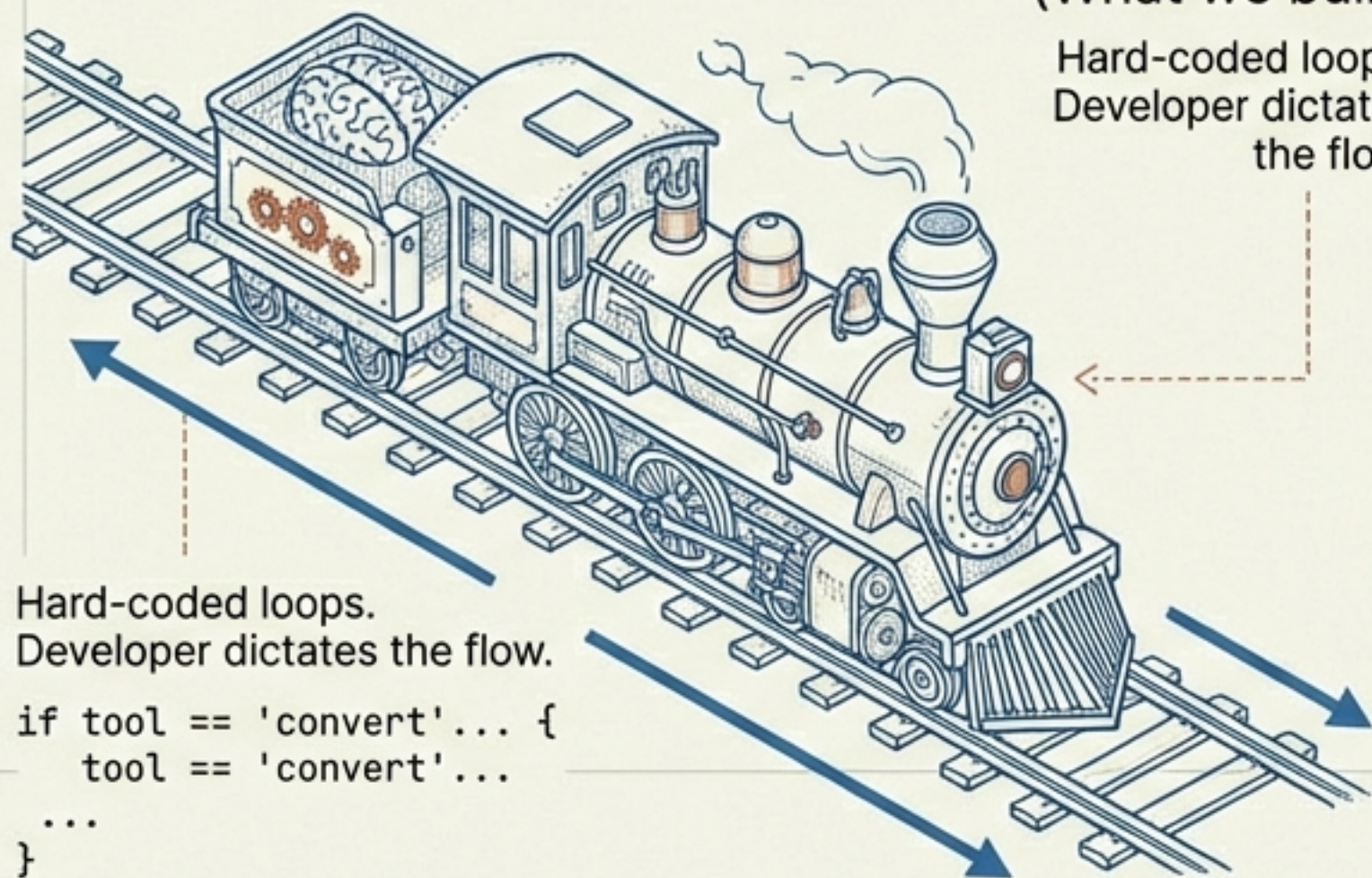
Takeaway: The LLM maintains the conversation state.
The Code maintains the data integrity.

The Agentic Horizon

Automation vs. Autonomy.

Tool Chain (What we built)

Hard-coded loops.
Developer dictates
the flow.

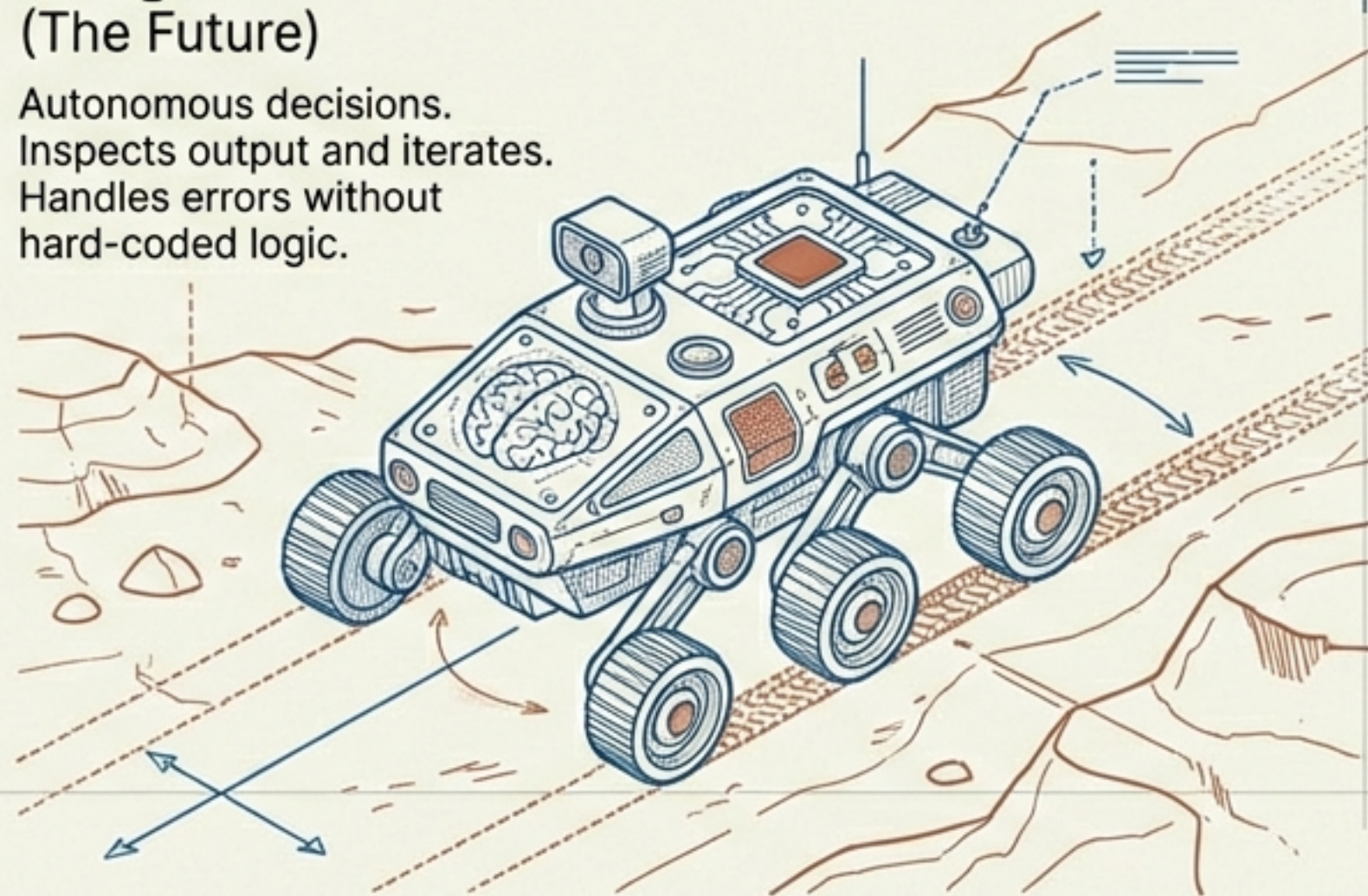


Hard-coded loops.
Developer dictates the flow.

```
if tool == 'convert'... {  
  tool == 'convert'...  
  ...  
}
```

AI Agent (The Future)

Autonomous decisions.
Inspects output and iterates.
Handles errors without
hard-coded logic.



**Tool Calling is the mechanical skill.
Agency is the strategic application.**

Further Exploration

Source Material:

“Tool Calling in LangChain” by CampusX

Next Steps:

1. Explore `langchain_core.tools` documentation.
2. Experiment with `bind_tools` on varying models.
3. Prepare for the transition from Chains to Agents.



Documentation
Helvetica Now Display